



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LESSON PLAN

For



AI Vibe Coding with Rust

TGS Ref No: TGS-2023039924

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 3.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	Not recorded	Initial release (Methodologies in C Programming)	Tertiary Infotech Academy
2.0	13 September 2026	Rebuilt as AI-Assisted C Programming for Arduino with 14 activities	Dr Alfred Ang
3.0	27 September 2026	Course retitled AI Vibe Coding with Rust: all topics, slides, Learner Guide, Lesson Plan, 14 verified Rust activity crates and assessments rewritten	Dr Alfred Ang

TABLE OF CONTENTS

Course Overview.....	4
Learning Outcomes.....	4
Daily Schedule.....	4
Topic-by-topic Breakdown.....	5
Topic 1: Rust Programming Fundamentals and Software Components (Slides 16-55).....	5
Activity 1: Turn a specification into a Rust function (Slides 41-45).....	5
Activity 2: Validate a product code with strings (Slides 46-50).....	5
Activity 3: Share inventory records with ownership and borrowing (Slides 51-55).....	6
Topic 2: AI Vibe Coding for Rust Application Development (Slides 56-95).....	6
Activity 4: Vibe-code a command parser (Slides 81-85).....	6
Activity 5: Organise a crate into modules (Slides 86-90).....	6
Activity 6: Load inventory JSON with Cargo dependencies (Slides 91-95).....	6
Topic 3: Rust Programming Controls, Functions and Features (Slides 96-136).....	7
Activity 7: Apply stock movements with enums and match (Slides 121-125).....	7
Activity 8: Track stock in a HashMap (Slides 126-130).....	7
Activity 9: Price mixed order lines with traits and iterators (Slides 131-135).....	7
Topic 4: Testing, Debugging and Integrating Rust Components (Slides 137-174).....	7
Activity 10: Parse CSV records with Result and custom errors (Slides 159-163).....	8
Activity 11: Debug a failing integration with tests (Slides 164-168).....	8
Activity 12: Integrate components into a tested command-line tool (Slides 169-173).....	8
Topic 5: AI-Assisted Rust Software Design and Documentation (Slides 175-207).....	8
Activity 13: Document an API with rustdoc and doctests (Slides 197-201).....	9
Activity 14: Design and document a reorder feature with AI (Slides 202-206).....	9
Resources Required.....	9
Assessment.....	9
Evidence and Source Notes.....	9

Course Overview

AI Vibe Coding with Rust (TGS-2023039924) is a two-day WSQ course mapped to TSC Software Design ICT-DES-3005-1.1. Learners build Rust components for the StockPilot stock service with an AI coding assistant, using the compiler, tests, clippy and rustfmt as the acceptance gate. Delivery: 9:30 AM to 6:30 PM; 14 training hours and 2 assessment hours. Courseware version 3.0.

Learning Outcomes

LO1: Determine basic software components using programming methodologies to meet functional specifications (K1/A1).

LO2: Apply programming methodologies and tools for software creation (K2/A2).

LO3: Select essential programming controls and features to meet software design requirements (K3/A3).

LO4: Examine the interoperability and functionality of programming components (K4/A4).

LO5: Generate programming design documentation aligned with user specifications (K5/A5).

Daily Schedule

Time	Minutes	Topic / Activity	Content and method	Slides
Day 1 9:30-11:30	120	Topic 1 / Activities 1-3	Admin (Slides 1-15) then Rust toolchain, types, functions, strings, ownership	Slides 16-55
Day 1 11:30-12:30	60	Topic 2 / Activity 4	Vibe-coding loop, bounded prompts, compiler as reviewer	Slides 56-95
Day 1 12:30-1:30	60	Lunch	Break / excluded	-
Day 1 1:30-3:30	120	Topic 2 / Activities 5-6	Modules, Cargo dependencies, fmt and clippy	Slides 56-95
Day 1 3:30-6:30	180	Topic 3 / Activities 7-9	match, collections, traits, generics and iterators	Slides 96-136
Day 2 9:30-12:30	180	Topic 4 / Activities 10-12	Errors, debugging, integration tests and CLI	Slides 137-174
Day 2 12:30-1:30	60	Lunch	Break / excluded	-
Day 2 1:30-4:30	180	Topic 5 / Activities 13-14 + briefing	rustdoc, doctests, traceability and design record	Slides 175-207

Time	Minutes	Topic / Activity	Content and method	Slides
Day 2 4:30-5:30	60	Written Assessment / K1-K5	Individual, open book	-
Day 2 5:30-6:30	60	Practical Performance / A1-A5	Individual, hands-on	-

Training contact: Topic 1 120 min, Topic 2 180 min, Topic 3 180 min, Topic 4 180 min, Topic 5 180 min = 840 min (14 h). Day 1: 480 training minutes. Day 2: 360 training minutes + 120 assessment minutes. Two 60-minute lunches are excluded. Activity minutes are hands-on practice inside each block; the remaining time covers concept teaching, demonstrations, AI-review discussion and feedback. Short breaks are taken within blocks as needed.

Topic-by-topic Breakdown

Topic 1: Rust Programming Fundamentals and Software Components (Slides 16-55)

K1/A1. Method: interactive lecture with worked Rust examples (divider Slide 16), live vibe-coding demonstration, then independent hands-on activities with trainer review.

Concept	Slides
A specification becomes a component contract	17-19
The Rust toolchain: rustup, cargo and rustc	20-22
Anatomy of a Cargo package	23-25
Variables are immutable by default	26-28
Choose integer types deliberately	29-31
Functions: the signature is the contract	32-34
String and &str: owned text and borrowed text	35-37
Ownership moves; borrowing lends	38-40

Activity 1: Turn a specification into a Rust function (Slides 41-45)

StockPilot must price an order line. The specification limits quantity to 1..10,000 and forbids a silently wrapped total. You create the Cargo package, write the function and prove it with tests. Mapping: K1/A1; 25 minutes; folder labs/activity-01-line-total.

Acceptance: cargo test reports 5 passed; cargo run prints "4 x 250 cents = 1000 cents". Trainer checks: signatures unchanged, boundary tests predicted from the specification, clippy and fmt clean, and a review record showing at least one AI proposal accepted, edited or rejected with a reason.

Activity 2: Validate a product code with strings (Slides 46-50)

Product codes arrive from scanners and spreadsheets with stray spaces and lower-case letters. StockPilot accepts only the SKU format AAA-9999 and normalises valid input to upper case. Mapping: K1/A1; 25 minutes; folder labs/activity-02-sku-validation.

Acceptance: cargo test reports 6 passed, including the non-ASCII and whitespace cases. Trainer checks: signatures unchanged, boundary tests predicted from the specification, clippy and fmt clean, and a review record showing at least one AI proposal accepted, edited or rejected with a reason.

Activity 3: Share inventory records with ownership and borrowing (Slides 51-55)

Three StockPilot functions handle the same item list differently: one only reads it, one changes one item and one consumes the list. You choose the parameter types that express those intentions.

Mapping: K1/A1 K3/A3; 25 minutes; folder labs/activity-03-ownership-borrowing.

Acceptance: cargo test reports 5 passed; the overflow test proves the item is unchanged. Trainer checks: signatures unchanged, boundary tests predicted from the specification, clippy and fmt clean, and a review record showing at least one AI proposal accepted, edited or rejected with a reason.

Topic 2: AI Vibe Coding for Rust Application Development (Slides 56-95)

K2/A2. Method: interactive lecture with worked Rust examples (divider Slide 56), live vibe-coding demonstration, then independent hands-on activities with trainer review.

Concept	Slides
The vibe-coding loop for Rust	57-59
Write a bounded prompt	60-62
The compiler is your first reviewer	63-65
Review AI-generated Rust before accepting it	66-68
Modules and visibility organise a crate	69-71
Cargo dependencies and Cargo.lock	72-74
fmt and clippy encode team practice	75-77
Set up VS Code for Rust vibe coding	78-80

Activity 4: Vibe-code a command parser (Slides 81-85)

Store staff will type commands such as "add ABC-0001 5". You describe the command language to an AI assistant, review the generated parser, and keep only code that passes the acceptance tests.

Mapping: K2/A2; 25 minutes; folder labs/activity-04-cli-commands.

Acceptance: cargo test reports 5 passed; cargo run -- add ABC-0001 5 prints Add { sku: "ABC-0001", qty: 5 }. Trainer checks: signatures unchanged, boundary tests predicted from the specification, clippy and fmt clean, and a review record showing at least one AI proposal accepted, edited or rejected with a reason.

Activity 5: Organise a crate into modules (Slides 86-90)

The reporting code has grown. You split StockPilot formatting into a money module and a report module, expose only what callers need, and keep the behaviour identical under test. Mapping: K2/A2; 25 minutes; folder labs/activity-05-modules-visibility.

Acceptance: cargo test reports 4 passed; tests import activity05::money and activity05::report. Trainer checks: signatures unchanged, boundary tests predicted from the specification, clippy and fmt clean, and a review record showing at least one AI proposal accepted, edited or rejected with a reason.

Activity 6: Load inventory JSON with Cargo dependencies (Slides 91-95)

The purchasing team exports inventory as JSON. You add serde and serde_json with Cargo, derive the data contract, and let the type system reject malformed records at the boundary. Mapping: K2/A2 K4/A4; 25 minutes; folder labs/activity-06-cargo-serde-json.

Acceptance: cargo test reports 6 passed; stock value of data/inventory.json is 65,948 cents. Trainer checks: signatures unchanged, boundary tests predicted from the specification, clippy and fmt clean, and a review record showing at least one AI proposal accepted, edited or rejected with a reason.

Topic 3: Rust Programming Controls, Functions and Features (Slides 96-136)

K3/A3. Method: interactive lecture with worked Rust examples (divider Slide 96), live vibe-coding demonstration, then independent hands-on activities with trainer review.

Concept	Slides
if and match are expressions	97-99
enum + match model every case	100-102
Loops: for, while and loop	103-105
Structs and impl blocks	106-108
Collections: Vec and HashMap	109-111
Traits and generics share behaviour	112-114
Closures and iterator chains	115-117
Option and Result are control features	118-120

Activity 7: Apply stock movements with enums and match (Slides 121-125)

Stock changes through receipts, shipments and manual adjustments. An enum names every movement and match forces each one to be handled, including the failure cases. Mapping: K3/A3; 30 minutes; folder labs/activity-07-match-control-flow.

Acceptance: cargo test reports 6 passed, covering every variant and every error. Trainer checks: signatures unchanged, boundary tests predicted from the specification, clippy and fmt clean, and a review record showing at least one AI proposal accepted, edited or rejected with a reason.

Activity 8: Track stock in a HashMap (Slides 126-130)

StockPilot needs fast lookup by SKU. You wrap a HashMap in an Inventory type so every change goes through methods that enforce the stock rules. Mapping: K3/A3; 30 minutes; folder labs/activity-08-hashmap-inventory.

Acceptance: cargo test reports 6 passed; low_stock(5) returns AAA-0001 and BBB-0002 only. Trainer checks: signatures unchanged, boundary tests predicted from the specification, clippy and fmt clean, and a review record showing at least one AI proposal accepted, edited or rejected with a reason.

Activity 9: Price mixed order lines with traits and iterators (Slides 131-135)

Orders can contain products and services (for example, device set-up hours). A Priced trait gives both one pricing contract, and iterator chains total and filter them without manual index loops. Mapping: K3/A3; 30 minutes; folder labs/activity-09-traits-iterators.

Acceptance: cargo test reports 5 passed; the mixed order totals 14,297 cents. Trainer checks: signatures unchanged, boundary tests predicted from the specification, clippy and fmt clean, and a review record showing at least one AI proposal accepted, edited or rejected with a reason.

Topic 4: Testing, Debugging and Integrating Rust Components (Slides 137-174)

K4/A4. Method: interactive lecture with worked Rust examples (divider Slide 137), live vibe-coding demonstration, then independent hands-on activities with trainer review.

Concept	Slides
Unit tests live beside the code	138-140
Integration tests check components together	141-143
Custom errors and the ? operator	144-146

Concept	Slides
Read a failing test like a detective	147-149
Debug with dbg!, focused tests and AI	150-152
Interoperability: arguments, files, exit codes	153-155
Serde: typed data at the boundary	156-158

Activity 10: Parse CSV records with Result and custom errors (Slides 159-163)

Stock files arrive with typos. Instead of crashing, StockPilot must report exactly which line is wrong and why, while still importing every valid line. Mapping: K4/A4; 30 minutes; folder labs/activity-10-error-handling.

Acceptance: cargo test reports 5 passed; data/stock.csv yields 2 records and errors on lines 3, 4 and 6. Trainer checks: signatures unchanged, boundary tests predicted from the specification, clippy and fmt clean, and a review record showing at least one AI proposal accepted, edited or rejected with a reason.

Activity 11: Debug a failing integration with tests (Slides 164-168)

The nightly stock summary is wrong: the total is too low and a correctly stocked item is flagged. The starter crate contains two real defects. You reproduce them with tests, diagnose them with the help of an AI assistant, fix them and keep regression tests. Mapping: K4/A4; 35 minutes; folder labs/activity-11-debug-integration.

Acceptance: Starter: 4 of 6 tests fail. After both fixes cargo test reports 6 passed. Trainer checks: signatures unchanged, boundary tests predicted from the specification, clippy and fmt clean, and a review record showing at least one AI proposal accepted, edited or rejected with a reason.

Activity 12: Integrate components into a tested command-line tool (Slides 169-173)

Operations staff will run StockPilot from a terminal and from scheduled scripts. You connect the library to a binary that reads a file, prints a report and returns exit codes that other programs can rely on. Mapping: K4/A4; 30 minutes; folder labs/activity-12-cli-integration.

Acceptance: cargo test reports 7 passed; cargo run -- ../data/stock.csv prints ITEMS 4, TOTAL \$526.55, LOW STOCK ABC-0003. Trainer checks: signatures unchanged, boundary tests predicted from the specification, clippy and fmt clean, and a review record showing at least one AI proposal accepted, edited or rejected with a reason.

Topic 5: AI-Assisted Rust Software Design and Documentation (Slides 175-207)

K5/A5. Method: interactive lecture with worked Rust examples (divider Slide 175), live vibe-coding demonstration, then independent hands-on activities with trainer review.

Concept	Slides
Doc comments become documentation	176-178
Doctests keep examples honest	179-181
Trace requirements to code and tests	182-184
Architecture and module maps	185-187
Record design decisions	188-190
AI-assisted documentation, human-verified	191-193
Hand over: README and maintenance notes	194-196

Activity 13: Document an API with rustdoc and doctests (Slides 197-201)

The maintenance team will own the reorder calculation. The starter code works but has no documentation. You write rustdoc comments whose examples are compiled and run as tests, and trace them to requirements. Mapping: K5/A5; 30 minutes; folder labs/activity-13-rustdoc-doctests.

Acceptance: cargo test runs 3 tests and 3 doctests; cargo doc --no-deps builds; clippy -D warnings is clean. Trainer checks: signatures unchanged, boundary tests predicted from the specification, clippy and fmt clean, and a review record showing at least one AI proposal accepted, edited or rejected with a reason.

Activity 14: Design and document a reorder feature with AI (Slides 202-206)

Capstone: purchasing wants weekly reorder suggestions. You agree requirements R1-R4, implement the feature with an AI assistant, and hand over a design record that traces every requirement to code and tests. Mapping: K5/A5 K4/A4; 30 minutes; folder labs/activity-14-design-capstone.

Acceptance: cargo test reports 6 passed and 1 doctest; docs/design.md traces R1-R4 to named tests. Trainer checks: signatures unchanged, boundary tests predicted from the specification, clippy and fmt clean, and a review record showing at least one AI proposal accepted, edited or rejected with a reason.

Resources Required

Current slide deck (AI-Vibe-Coding-with-Rust-v3.0), Learner Guide, and the course repository with 14 activity folders. Per learner: Windows or Mac laptop, Rust stable via rustup (with clippy and rustfmt), Visual Studio Code with rust-analyzer, an AI coding assistant, and internet access (Activity 6 downloads the serde crates). Trainer: projector, the same toolchain, and the restricted assessment answer keys (not distributed to learners).

Assessment

Day 2, 4:30-5:30 PM: Written Assessment (SAQ), 60 minutes, 5 open-ended questions covering K1-K5. 5:30-6:30 PM: Practical Performance (PP), 60 minutes, 5 hands-on tasks covering A1-A5 using the class Rust crates. Both are open book and individual. Learners download papers and upload answers on the LMS (<https://lms-tms.tertiaryinfotech.com/>). Outcome: Competent (C) or Not Yet Competent (NYC); re-assessment and appeal follow the ATO procedure.

Evidence and Source Notes

All slide references in this plan are read from the built slide_map.json of the current deck. The 14 activity crates were verified with rustc 1.93.1 (01f6ddf75 2026-02-11) (build/qa/labs-verification.json). Sources are listed in the Learner Guide.